22

Using Views

In this chapter, you'll learn exactly what views are, how they work, and when they should be used. You'll also see how views can be used to simplify some of the SQL operations performed in earlier chapters.

Understanding Views

Note

This Chapter Requires MySQL 5 or Later Support for views was added to MySQL 5. Therefore, this chapter is applicable to MySQL 5 or later only.

Views are virtual tables. Unlike tables that contain data, views simply contain queries that dynamically retrieve data when used.

The best way to understand views is to look at an example. Back in Chapter 15, "Joining Tables," you used the following `SELECT` statement to retrieve data from three tables:

Input

```
SELECT cust_name, cust_contact
FROM customers, orders, orderitems
WHERE customers.cust_id = orders.cust_id
  AND orderitems.order_num = orders.order_num
  AND prod_id = 'TNT2';
```

That query was used to retrieve the customers who had ordered a specific product. Anyone needing this data would have to understand the table structure, as well as how to create the query and join the tables. To retrieve the same data for another product (or for multiple products), the tables would all need to be joined, and the last `WHERE` clause would have to be modified.

Now imagine that you could wrap that entire query, complete with all the table relationships defined, in a virtual table called `productcustomers`. You could then simply use the following to retrieve the same data:

Input

```
SELECT cust_name, cust_contact
FROM productcustomers
WHERE prod_id = 'TNT2';
```

This is where views come into play. `productcustomers` is a view, and as a view, it does not contain any actual columns or data, as a table would. Instead, it contains a SQL query—the same query used previously to join the tables.

Why Use Views

You've already seen one use for views. Here are some other common uses:

- To reuse SQL statements.
- To simplify complex SQL operations. After a query is written, it can be reused easily, without requiring the user to know the details of the underlying query.
- To expose parts of a table instead of the complete table.
- To secure data. Users can be given access to specific subsets of tables instead of to entire tables.
- To change data formatting and representation. Views can return data formatted and presented differently than in their underlying tables.

For the most part, after views are created, they can be used in the same way as tables. You can perform `SELECT` operations, filter and sort data, join views to other views or tables, and possibly even add and update data. (There are some restrictions on this last item. More on that in a moment.)

The important thing to remember is that views provide views into data stored elsewhere. Views contain no data themselves, and the data they return is retrieved from other tables. When data is added or changed in those tables, the views return the changed data.

Caution

Performance Issues Because views contain no data, any retrieval needed to execute a query must be processed every time the view is used. If you create complex views with multiple joins and filters, or if you nest views, you may find that performance is dramatically degraded. Be sure you test execution before deploying applications that use views extensively.

View Rules and Restrictions

Here are some of the most common rules and restrictions governing view creation and usage:

- Like tables, views must be uniquely named. (That is, a view cannot be given the same name as any other table or view.)
- There is no limit to the number of views that can be created.
- To create views, you must have security access. This is usually granted by the database administrator.
- Views can be nested; that is, a view may be built using a query that retrieves data from another view.
- `ORDER BY` can be used in a view, but it will be overridden if `ORDER BY` is also used in the `SELECT` that retrieves data from the view.
- Views cannot be indexed, and they cannot have triggers or default values associated with them.
- Views can be used in conjunction with tables (for example, to create a `SELECT` statement that joins a table and a view).

Using Views

Now that you know what views are (and the rules and restrictions that govern them), let's look at view creation:

- Views are created using the `CREATE VIEW` statement.
- To view the statement used to create a view, use `SHOW CREATE VIEW` *`viewname`*`;`.
- To remove a view, use the `DROP` statement. The syntax is simply `DROP VIEW` *`viewname`*`;`.
- To update a view, you can use the `DROP` statement and then the `CREATE` statement again, or you can just use `CREATE OR REPLACE VIEW`, which creates the view if it does not exist and replaces it if it does.

Using Views to Simplify Complex Joins

One of the most common uses of views is to hide complex SQL, and this often involves joins. Look at the following statement:

Input

```
CREATE VIEW productcustomers AS
SELECT cust_name, cust_contact, prod_id
FROM customers, orders, orderitems
WHERE customers.cust_id = orders.cust_id
  AND orderitems.order_num = orders.order_num;
```

Analysis

This statement creates a view named `productcustomers`, which joins three tables to return a list of all customers who have ordered any product. If you use `SELECT * FROM productcustomers`, you get a list of every customer who has ordered anything.

To retrieve a list of customers who have ordered product TNT2, you can use the following:

Input

```
SELECT cust_name, cust_contact
FROM productcustomers
WHERE prod_id = 'TNT2';
```

Output

```
+----------------+-----------------+
| cust_name      |  cust_contact   |
+----------------+-----------------+
| Coyote Inc.    | Y Lee           |
| Yosemite Place | Y Sam           |
+----------------+-----------------+
```

Analysis

This statement retrieves specific data from the view by issuing a `WHERE` clause. When MySQL processes the request, it adds the specified `WHERE` clause to any existing `WHERE` clauses in the view query so the data is filtered correctly.

As you can see, views can greatly simplify the use of complex SQL statements. By using a view, you can write the underlying SQL once and then reuse it as needed.

Tip

Creating Reusable Views It is a good idea to create views that are not tied to specific data. For example, the view created in this example returns customers for all products, not just product TNT2 (for which the view was first created). By expanding the scope of a view, you enable it to be reused, making it even more useful. You also eliminate the need to create and maintain multiple similar views.

Using Views to Reformat Retrieved Data

As mentioned previously, another common use of views is for reformatting retrieved data. The following `SELECT` statement (from Chapter 10, "Creating Calculated Fields") returns the vendor name and location in a single combined calculated column:

Input

```
SELECT Concat(RTrim(vend_name),
              ' (', RTrim(vend_country), ')')
       AS vend_title
FROM vendors
ORDER BY vend_name;
```

Output

```
+------------------------+
| vend_title             |
+------------------------+
| ACME (USA)             |
| Anvils R Us (USA)      |
| Furball Inc. (USA)     |
| Jet Set (England)      |
| Jouets Et Ours (France)|
| LT Supplies (USA)      |
+------------------------+
```

Now suppose that you regularly need results in this format. Rather than perform the concatenation each time it is needed, you can create a view and use that instead. To turn this statement into a view, you can use the following:

Input

```
CREATE VIEW vendorlocations AS
SELECT Concat(RTrim(vend_name),
       ' (', RTrim(vend_country), ')')
       AS vend_title
FROM vendors
ORDER BY vend_name;
```

Analysis

This statement creates a view using exactly the same query as the previous `SELECT` statement. To retrieve the data to create all mailing labels, simply use the following:

Input

```
SELECT *
FROM vendorlocations;
```

Output

```
+------------------------+
| vend_title             |
+------------------------+
| ACME (USA)             |
| Anvils R Us (USA)      |
| Furball Inc. (USA)     |
| Jet Set (England)      |
| Jouets Et Ours (France)|
| LT Supplies (USA)      |
+------------------------+
```

Using Views to Filter Unwanted Data

Views are also useful for applying common `WHERE` clauses. For example, you might want to define a `customeremaillist` view so it filters out customers without email addresses. To do this, you can use the following statement:

Input

```
CREATE VIEW customeremaillist AS
SELECT cust_id, cust_name, cust_email
FROM customers
WHERE cust_email IS NOT NULL;
```

Analysis

Obviously, when sending email to a mailing list, you'd want to ignore users who have no email address. The `WHERE` clause here filters out rows that have `NULL` values in the `cust_email` columns so they will not be retrieved.

You can now use the `customeremaillist` view for data retrieval just as you would any table. Consider this example:

Input

```
SELECT *
FROM customeremaillist;
```

Output

```
+---------+---------------+-----------------------+
| cust_id | cust_name     | cust_email            |
+---------+---------------+-----------------------+
|   10001 | Coyote Inc.   | ylee@coyote.com       |
|   10003 | Wascals       | rabbit@wascally.com   |
|   10004 | Yosemite Place | sam@yosemite.com     |
+---------+---------------+-----------------------+
```

Note

`WHERE` Clauses If a `WHERE` clause is used when retrieving data from a view, the two sets of clauses (the one in the view and the one passed to it) will be combined automatically.

Using Views with Calculated Fields

Views are exceptionally useful for simplifying the use of calculated fields. The following is a `SELECT` statement introduced in Chapter 10 that retrieves the order items for a specific order and calculates the expanded price for each item:

Input

```
SELECT prod_id,
       quantity,
       item_price,
```

```
       quantity*item_price AS expanded_price
FROM orderitems
WHERE order_num = 20005;
```

► Output

```
+---------+----------+------------+----------------+
| prod_id | quantity | item_price | expanded_price |
+---------+----------+------------+----------------+
| ANV01   |       10 | 5.99       | 59.90          |
| ANV02   |        3 | 9.99       | 29.97          |
| TNT2    |        5 | 10.00      | 50.00          |
| FB      |        1 | 10.00      | 10.00          |
+---------+----------+------------+----------------+
```

To turn this into a view, use the following:

► Input

```
CREATE VIEW orderitemsexpanded AS
SELECT order_num,
       prod_id, quantity,
       item_price,
       quantity*item_price AS expanded_price
FROM orderitems;
```

To retrieve the details for order 20005 (the previous output), use the following:

► Input

```
SELECT *
FROM orderitemsexpanded
WHERE order_num = 20005;
```

► Output

```
+-----------+---------+----------+------------+----------------+
| order_num | prod_id | quantity | item_price | expanded_price |
+-----------+---------+----------+------------+----------------+
|   20005   | ANV01   |       10 | 5.99       | 59.90          |
|   20005   | ANV02   |        3 | 9.99       | 29.97          |
|   20005   | TNT2    |        5 | 10.00      | 50.00          |
|   20005   | FB      |        1 | 10.00      | 10.00          |
+-----------+---------+----------+------------+----------------+
```

As you can see, views are easy to create and even easier to use. When used correctly, views can greatly simplify complex data manipulation.

Updating Views

All of the views thus far have been used with `SELECT` statements. But can view data be updated? It depends.

As a rule, yes, views are updatable (that is, you can use `INSERT`, `UPDATE`, and `DELETE` on them). Updating a view updates the underlying table. (Recall that the view has no data

of its own.) If you add or remove rows from a view you are actually removing them from the underlying table.

But not all views are updatable. Basically, if MySQL is unable to correctly ascertain the underlying data to be updated, updates (including insertions and deletions) are not allowed. In practice, this means that if any of the following are used, you will not be able to update the view:

- Grouping (using `GROUP BY` and `HAVING`)
- Joins
- Subqueries
- Unions
- Aggregate functions (`Min()`, `Count()`, `Sum()`, and so forth)
- `DISTINCT`
- Derived (calculated) columns

In other words, many of the examples used in this chapter would not be updatable. This might sound like a serious restriction, but it really isn't because views are primarily used for data retrieval.

Tip

Use Views for Retrieval As a rule, you should use views for data retrieval (`SELECT` statements) and not for updates (`INSERT`, `UPDATE`, and `DELETE` statements).

Summary

Views are virtual tables. They do not contain data; rather, they contain queries that retrieve data as needed. Views provide a level of encapsulation around MySQL `SELECT` statements and can be used to simplify data manipulation as well as to reformat or secure underlying data.

Challenges

1. Create a view called `VendorProducts` that joins `Vendors` and `Products` tables. Use a `SELECT` to make sure you have the right data.
2. Create a view called `CustomersWithOrders` that contains all of the columns in `Customers` but includes only customers who have placed orders. Here's a hint: You can use `JOIN` on the `Orders` table to filter just the customers you want. Then use `SELECT` to make sure you have the right data.